

Erőforrás kezelés, holtpont

Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek és
+ URL linkek alapján készült el

Témakörök

- Erőforrás fogalma, csoportosítása
- Erőforrások menedzselése
- Erőforrás menedzselése - Kölcsönös kizárás
- Kölcsönös kizárás - fogalmak
- Kölcsönös kizárás - megoldások

Témakörök

- Holtpont
- Holtpont - kialakulásának szükséges feltételei
- Holtpont stratégiák
- Bankár algoritmus
- Mintapéldák

OS fogalma - ismételés

„1. Erőforrás menedzser (Resource Manager)

- Hatékonyá, biztonságossá, igazságossá teszi a géphasználatot.

2. Kiterjesztett gép (Extended Machine)

- Kényelmessé, "felhasználóbarát" jellegűvé teszi a géphasználatot.

3. ISO szabvány szerinti (nemzetközi szabványosítási szervezet definíciója)

OS fogalma - erőforrások menedzser - ismétlés

Erőforrások menedzselése

- *Erőforrások:*

- hardver (processzor, tárák stb.)
- szoftver (alkalmazások, adatbázisok stb.)
- emberi (felhasználók, rendszergazdák stb.)

OS fogalma - erőforrások menedzser - ismétlés

- *Menedzselés:*

- *Elosztja az erőforrásokat a processzek között (diszkek, perifériák, tárak, processzorok)*
- *Védelem (kölcsonös kizárás a kritikus szakaszok)*
- *Nyilvántartás (statisztikát készít az erőforrások használatáról)*

Erőforrás fogalma

Erőforrás fogalma

„Minden, ami egy *processz végrehajtásához* szükséges.
Az erőforrásokat különféleképpen csoportosíthatjuk.

1. Csoportosítás

- *Hardver erőforrások*, a processzor, memória, nyomtató és az egyéb perifériák.
- *Szoftver erőforrások* programok, adatállományok, adatbázisok stb.
- *Emberi erőforrások* (felhasználók, rendszergazdák stb.)

Erőforrások csoportosítása

2. Csoportosítás

- *Hagyományos*: nyomtatók, szövegszerkesztők.
- *Operációs rendszer által létrehozott*: fájl leíró táblák, lemez adatblokkok, pufferek, folyamatvezérlő blokkok (PCB), laptáblák, szegmensleíró táblák.

Erőforrások csoportosítása - megszakítható

3. Csoportosítás: *megszakítható, nem megszakítható*

Megszakítható (preemptív)

A processztől az erőforrás *károsodása nélkül elvehető* – OS rendelkezik felette.

Példa: a processz és a memória, hiszen a *folyamatleíró blokk* minden olyan információt tartalmaz, mely szükséges az eredeti állapot visszaállításához.

Erőforrások csoportosítása - nem megszakítható

Nem megszakítható (non-preemptív)

Az erőforrás használat félbeszakítása esetén a *processz* vagy *erőforrás* sérülhet!

Akkor szükséges, ha egy processz egy erőforráson olyan műveleteket végez, mint pl.: nyomtató, mágnesszalagos egység.

Erőforrások menedzselése - feladata

Feladata:

- gondoskodik a *számítógép erőforrásainak* (a futó processzek igényei alapján történő) *hatékony*,
- *gazdaságos elosztásáról*, ill.
- az erőforrások használatáért vívott *versenyhelyzetek kezeléséről*.

Erőforrások fajtái - megosztható

A processzek életük során *nem csak a CPU erőforrásért vetélkednek*, hanem *más erőforrásokért* is.

Erőforrások fajtái:

Megosztható

Egy időben több processz is használhatja vagy bármikor elvehető a használótól és átadható más processznek, majd vissza is adható.

Erőforrások fajtái – nem megosztható

Nem megosztható

Egy időben *csak egy processz* használhatja és *nem vehető el* tőle - amíg használja.

Erőforrások menedzselése - nem megosztható

Nem megosztható erőforrások kezelése

1. Ütemező használata:

Teljesen kontrollálható kezelés, viszont teljesítmény probléma, és *csak konkrét erőforrásoknál használható* (az OS csak processzekre használja, ill. I/O).

2. Visszavezetni *megosztható erőforrás* használatára a használatot: pl.: nyomtatásnál.

Nem használható bármilyen erőforrásfajtánál.

Erőforrások menedzselése - nem megosztható

3. *Kölcsönös kizárás biztosítása:* bármilyen erőforrásra használható, megvalósítás problémák.

Erőforrás menedzselése - Kölcsönös kizárás

Kölcsönös kizárás

A közös erőforrásokért vetélkedő processzek közül egy és csakis egy kapja meg a jogot az erőforrás használatára.

A kölcsönös kizárásnak vannak: módszerei, eszközei, technikái.

Kölcsönös kizárás - problémák

Megvalósítás problémája:

Hogyan lehet olyan megoldást készíteni, amely nem csak az OS erőforrásaira használható, hanem tetszőleges szoftver erőforrásra is?

- Konkurencia (vetélytárs) probléma.
- *Több kölcsönös kizárást igénylő erőforrás* használatából felmerülő probléma: **holtpont**

Kölcsönös kizárás – megoldás fogalmai

Megoldással kapcsolatos fogalmak:

- Belépési szakasz.
- Kritikus szakasz.
- Kilépési szakasz.

Kölcsönös kizárás - fogalmak

Belépési szakasz: a processzen belül az a kódrész, ahol kéri az engedélyt a kritikus szakaszba való belépésre.

Kritikus szakasz: a processzen belüli kódrész, melyen belül a kölcsönös kizárást meg kell valósítani, vagy amire az ütemezést meg kell valósítani.

*Egy erőforrás kizárólagos használatának **időszak**a.*

Kölcsönös kizárás - fogalmak

Kilépési szakasz: a processzen belül az a kódrész, ahol *elhagyja* a processz a *kritikus szakaszt*. (Visszaadja az engedélyt.)

A processzeknek természetesen lehetnek *nem kritikus szakaszaik* is.

Kölcsönös kizárás - követelmények

Milyen követelményeket támasztunk a kölcsönös kizárás megvalósításával szemben?

Kívánalmak a *megoldásához!*

- 1.** *Biztonsági* (safety) kíváncalom.
- 2.** *Előrehaladási* (progress) kíváncalom.
- 3.** *Korlátozott várakozási* (bounded waiting) kíváncalom.
- 4.** *Hardver és platform* kíváncalom.
- 5.** *Teljesítmény* kíváncalom.

Kölcsönös kizárás - követelmények

1. Biztonsági (safety) *kíváncalom*: ha egy processz *kritikus szakaszában* fut, akkor más processz *ne léphessen be kritikus szakaszába*.

2. Előrehaladási (progress) *kíváncalom*: a futó processz *ne befolyásolja mások belépését, amely általában nem kritikus szakaszban és nem belépési szakaszban*.

Kölcsönös kizárás - követelmények

3. Korlátozott várakozási (bounded waiting) kívánalom: ha egy processz bejelentette igényét a belépésre, de még nem lépett be, *korlátozzuk azt, hogy egy másik processz hányszor léphet be.*

Kölcsönös kizárás - követelmények

4. Hardver és platform kíváncalom: *ne legyenek előfeltételeink a hardverre* (sem a CPU-k típusára, számára, sem a sebességükre), a processzek számára, relatív sebességükre, az operációs rendszer ütemezésére stb.

5. Teljesítmény kíváncalom: A processzek várakozás közben ne foglalják a CPU-t, azaz *ne legyen tevékeny várakozás*.

Kölcsönös kizárás – megoldások

1. Megszakítás letiltás

2. Egyszerű zárolás változó

3/a. Egyszerű váltogatás (ki van soron)

3/b. Peterson algoritmus

4/a. TSL instrukció (Test and Set Lock)

4/b. CAS instrukció (Compare and Swap)

Kölcsönös kizárás – megoldások

5. Szemafor

6. Monitor mechanizmus

7. RCU (read-copy-update)

Kölcsönös kizárás - Megszakítás letiltás

1. Megszakítás letiltás

A megoldás *lényege*: a *belépési szakaszban* letiltunk minden megszakítást, a *kilépési szakaszban engedélyezük* azokat.

A megoldás *hátránya*: Csak egy CPU-ra jó (4. követelmény!).

Veszélyes: hiba esetén *deadlock*! Csak nagyon letesztelt kernel kódokban!

A programozó kezébe nem adható eszköz!

Kölcsönös kizárás - Egyszerű zárolás változó

2. Egyszerű zárolás változó:

- Adott egy *osztott lock* változó, kezdeti értéke *false*, ami *tesztelhető* és beállítható (false és true értékeket vehet fel).
- Ha egy processz be akar lépni *kritikus szakaszába*, teszteli a *lock* változót.
- Ha az *false*, beállítja *true-ra* és *belép a kritikus szakaszba*.
- Ekkor a ciklusban teszteli a változót (busy waiting), és megvárja, míg valaki lenullázza.

Kölcsönös kizárás - Egyszerű zárolás változó

- Fontos követelmény, hogy a *tesztelés* és *beállítás* között a vezérlést ne vehessék el a processztől, különben megsértjük az **1.** számú követelményt.

```
processz-i() {  
    while (true) {  
        nem-kritikus-szakasz1();  
        while (lock) { nop(); }  
        lock = true;  
        kritikus-szakasz();  
        lock = false;  
        nem-kritikus-szakasz2();  
    }  
}
```

Kölcsönös kizárás - Egyszerű zárolás változó

Sérül az **1.** szabály (biztonság):

A *lock* változó használata szinten kölcsönös kizárást igényelne!

- Sérül az **5.** szabály (teljesítmény)

```
processz-i() {  
    while (true) {  
        nem-kritikus-szakasz1();  
        while (lock) { nop(); }  
        lock = true;  
        kritikus-szakasz();  
        lock = false;  
        nem-kritikus-szakasz2();  
    }  
}
```

Kölcsönös kizárás - Egyszerű váltogatás

3. Egyszerű váltogatás (ki van soron)

A megoldáshoz szükséges egy osztott (közös) **turn** változó.

Jelentése : ki van soron. Csak felváltva lehet belépni.

Ez a változó nyomonköveti, azt hogy ki következik.

A kritikus szakaszba való belépés while ciklusa!

Amíg teljesül a while feltétele, a vezérlés menete tevékeny várakozásban a ciklusban marad!

Kölcsönös kizárás - Egyszerű változtatás

- A `nop()` azt jelzi, ne *csináljon semmit*, azaz a ciklus magja egy *no-operation instrukció*.
- Sérül a **2. szabály** (előre haladás): a nem kritikus szakaszában is feltartja a másikat.

Ez azt jelenti, hogy egy processz egymás után kétszer csak akkor léphet a kritikus szekciójába, ha közben a másik is már belépett.

Kölcsönös kizárás - Egyszerű változtatás

- Sérül a **4.** szabály (HW/arch): csak két processzre.
- Sérül az **5.** szabály (telj.).

Hátránya: nem követi, hogy egy processz érdekelt-e vagy sem.

Kölcsönös kizárás - Peterson algoritmus

3/b. Peterson algoritmus: lényegében egymás váltogatása az *érdekeltség figyelembe vételével*.

Az osztott (közös) `turn` változó, *nyomon követi*, hogy ki következik.

Az osztott `enter` tömb jelzi, ki érdekelt a *kritikus szakasz* elérésében.

- Sérül az **5.** szabály (telj.)

Kölcsönös kizárás - Peterson algoritmus

Peterson algoritmus: legyen az, hogy egyik processz sincs *kritikus szakaszában*, de az **i-edik** be akar lépni.

Beállítja *saját érdekelttségét*, és a másik *következőségét*.

Utána következik a *tevékeny várakozás* `while` ciklusára.

Mivel a másik nem érdekelt, így beléphet a *kritikus szakaszba*.

Kölcsönös kizárás - Peterson algoritmus

Ha most a *j*-edik szeretne belépni a `while`-jában, akkor meg kell várnia, míg az *i* processz a *kritikus szakaszából* kilépve *megszünteti az érdekeltségét*.

shared turn, erd[N];

```
processz-i() {  
  while (true) {  
    nem-kritikus-szakasz1();  
    erd[i]=true; turn=j;  
    while(erd[j]&&turn==j) ;  
    kritikus-szakasz();  
    erd[i]=false;  
    nem-kritikus-szakasz2();  
  }  
}
```

```
processz-j() {  
  while (true) {  
    nem-kritikus-szakasz1();  
    erd[j]=true; turn=i;  
    while(erd[i]&&turn==i) ;  
    kritikus-szakasz();  
    erd[j]=false;  
    nem-kritikus-szakasz2();  
  }  
}
```

Kölcsönös kizárás - Peterson algoritmus

Ha mindkét processz kb. egyidőben lépne be:

- akkor *mindkettő bejelenti érdekeltségét,*
- majd mindkettő *bejegyzi a turn-be* a másik számát.
- amelyiknek ez utoljára sikerült, az vesztett!

A másiknak a *while*-ja "sikertelen" lesz, azaz tovább léphet.

Kölcsönös kizárás - TSL instrukció

4. TSL instrukció (Test and Set Lock)

- Egy *processzor utasítás*, amely egy *megszakíthatatlan* (atomi) lépésben leellenőrzi *a zárolásváltozót* és *beállítja*.
- Sérül a **4.** követelmény (HW/arch)
- Sérül az **5.** követelmény (telj.)

Kölcsönös kizárás - TSL instrukció

Többprocesszoros rendszerekhez terveztek, létezik egy atomi *test-and-set-lock* (TSL) instrukció.

Működése:

- Betölt egy memóriacímen lévő értéket egy regiszterbe, és egy `true` értéket tesz a memória rekeszbe, majd a behozott értékkel visszatér.

Kölcsönös kizárás - TSL instrukció

- A regiszterbe *hozás, a letárolás műveletére és visszatérésre* garantált, hogy "oszthatatlan", így más processzor sem érheti el a *memória rekeszt*, amíg az *instrukció végre nem hajtódik*.
- Általában a TSL-t végrehajtó CPU blokkolja a buszt, amíg a TSL-t végrehajtja.

Kölcsönös kizárás - TSL instrukció

A kritikus szakasz védelmét biztosító kódrészletet és a TSL instrukció "forgatókönyvét" is.

shared lock=false;

```
processz-i() {  
    while (true) {  
        nem-kritikus-szakasz1();  
        while(tsl(&lock)) nop();  
        kritikus-szakasz();  
        lock=false;  
        nem-kritikus-szakasz2();  
    }  
}
```

```
int tsl(*lock) {  
    register int temp;  
    temp=*lock;  
    *lock=true;  
    return temp;  
} // ez megszakíthatatlan  
// instrukcio kell legyen
```

Kölcsönös kizárás - CAS instrukció

4/b. CAS instrukció (Compare and Swap)

- Egy *processzor utasítás*, amely egy *megszakíthatatlan* (atomi) *lépésben összehasonlít és kicserél értékeket*.
- pl. x86 CMPXCHG instrukció

Részletes leírás a jegyzetbe!

Kölcsönös kizárás - Szemafor

5. Szemafor



Kölcsönös kizárás - Szemafor

5. Szemafor

- Dijkstra (1965) (holland)
- Csak az elvi alapokat adta, a megvalósításról nem rendelkezett.
- Alkalmas *szinkronizáció megvalósítására* is.

Kölcsönös kizárás – Szemafor működése

A szemafor:

- *pozitív egészet tartalmazó közös változó és*
- *egy várakozási sor együttese.*

A szemaforon *két atomi* (azaz *nem megszakítható*) operáció végezhető:

- DOWN - ezen blokkolódhat a hívója.
- UP művelet (P és V primitív) - ez szignál, hogy felébredjen egy blokkolódott processz.

Kölcsönös kizárás - Szemafor

5. Szemafor

DOWN - *P: Passeren*

- Ha a változó nagyobb, mint nulla ($s > 0$): a változó értéket csökkenti eggyel.
- Ha nulla ($s = 0$): a hívó processz *a várakozó sorba kerül* (blokkolódik).

Pseudo kóddal:

Kölcsönös kizárás - Szemafor

UP (*V: Vrijgeven [vrehéfen]*)

A változó értéke *növekszik eggyel* és ha *nulla volt akkor* jelez egy *blokkolódott processznek*, hogy *felébredhet*.

Pszedó kóddal:

Kölcsönös kizárás - Szemafor

A kölcsönös kizárás megvalósítása szemaforral:

- Az **1.** (safety), **2.**, (progress) **4.** (hw) követelményt biztos, hogy *kielégíti*.
- A **3.** (előrehaladás) kielégítése az **UP** ébresztés megvalósításától függ.
- Az **5.** kielégítése a várakozás megvalósításától függ.

Kölcsönös kizárás - Szemafor

A várakozás lehetséges megvalósításai:

1. A várakozás *tevékeny várakozás*, ebből ébresztés signál-lal (spinlock szemafor). Az **5. követelmény** sérülhet.

Előnye lehet, hogy nincs kontextus váltás (Alkalmazás, ha a CS költsége nagyobb, mint a várakozásé)

Kölcsönös kizárás - Szemafor

2. *A várakozás blokkolással (wait állapotátmenet $\rightarrow$ CS), az ébresztés signal-lal (signal állapotátmenet).*

Minden követelmény kielégít *(legalábbis a 3. követelmény (korl. vár.) kielégítését átruházza a scheduler-re).*

Kölcsönös kizárás - Szemafor

Atomiság lehetséges megvalósításai: bármelyik korábbi módszerrel.

Lehetséges megszakítás letiltással is, hiszen ez OS (kernel) kód!
(ha rendelkezésre áll, akkor persze érdemes TSL vagy CAS instrukciót használni)

Kölcsönös kizárás - Bináris szemafor

Kétértékű változót használ (0/1). Tehát:

DOWN: A változó, ha hamis (0), akkor igazra (1) állítja, ha igaz, akkor várakozás.

UP: A változót hamisra állítja és ha van várakozó akkor *ébreszti*.

Kölcsönös kizáráshoz elegendő. A nem bináris szemaforok használata inkább szinkronizációra.

Kölcsönös kizárás – Monitor mechanizmus

6. Monitor mechanizmus

- Magasabb szintű *kölcsönös kizárást és szinkronizációt* biztosító mechanizmus: *eljárások, változók, adatstruktúrák speciális formájú gyűjteménye.*
- A megvalósítás *szemaforok használatával*, de lehetne más módszer is.

Kölcsönös kizárás - Monitor mechanizmus

A processzek hívhatják a *monitor eljárásait*, de

- nem férnek a *monitor belső adatstruktúráihoz*,
- ill. egy időben csak *egy processz használhat egy monitort*.

A *fordító biztosítja* a monitorba való belépésre a kölcsönös kizárást.

Kölcsönös kizárás - Monitor mechanizmus

Ha egy processz hív egy monitorban *lévő eljárást*, az első néhány *instrukció ellenőrzi*, hogy a *processz aktív-e a monitorban*.

Ha igen, a hívó *blokkolódik*, míg a másik elhagyja a monitort.

Kölcsönös kizárás - Monitor mechanizmus

Egyszerű monitor példa:

- Csak a *producer* és a *consumer* eljárások láthatók.
- Az *i* és *c* belső változókat csak a belső eljárások kezelhetik.
- Legfeljebb egy eljárás lehet aktív: vagy a *producer*, vagy a *consumer*.

```
monitor example
  integer i;
  condition c;

  procedure producer();

  end;

  procedure consumer();

  end
end monitor;
```

Kölcsönös kizárás - Monitor mechanizmus

Egy általános szintaktika:

```
monitor (változó)
    kritikus szakasz
end monitor
```

Java szintaktika:

```
synchronized (refváltozó) {
    kritikus szakasz
}
```

Kölcsönös kizárás - RCU (read-copy-update)

7. RCU (read-copy-update)

- Nincs blokkolás! Gyors, nem áll meg a folyamat.
- Láncolt listás adatszerkezetekhez (pl. processz lista)
- Egy másolat készül az adat(szerkezet)ről és külön kell őket karbantartani (több példány)
- Szinkronban frissül mindig.
- Szükség esetén bevárja a frissítéseket.

Holtpont

Holtpont

Holtpont fogalma

„A **holtpont** (deadlock) az az állapot, amely akkor következhet be, amikor *két (vagy több) processz* egyidejűleg verseng erőforrásokért, és egymást kölcsönösen blokkolják.

Tegyük fel, hogy

- *P processz* kizárólagos használatra kéri az **X** és **Y** erőforrásokat, és azokat ebben a sorrendben kívánja használni.
- *Q processz* is kéri az **Y** és **X** erőforrásokat, ebben a sorrendben.

Holtpont fogalma

- Ha *P processz* megszerezte az **X** erőforrást,
- *Q processz*, pedig az **Y**-t, akkor egyik sem tud továbblépni,

Mert, mindkettőnek éppen arra volna szüksége, amit a másik foglal: ez a *holtpont helyzet*. „ (forrás: Vadász D.: OS)

Holtpont – Következmény - példa

Következmény processzek esetén: a holtpontban lévő processzek nem tudnak folytatódni.

Az általuk foglalt erőforrások nem tudnak felszabadulni.

Az ezeket igénylő processzek holtpontba kerülnek.

Holtpont keletkezése

Kialakulhat:

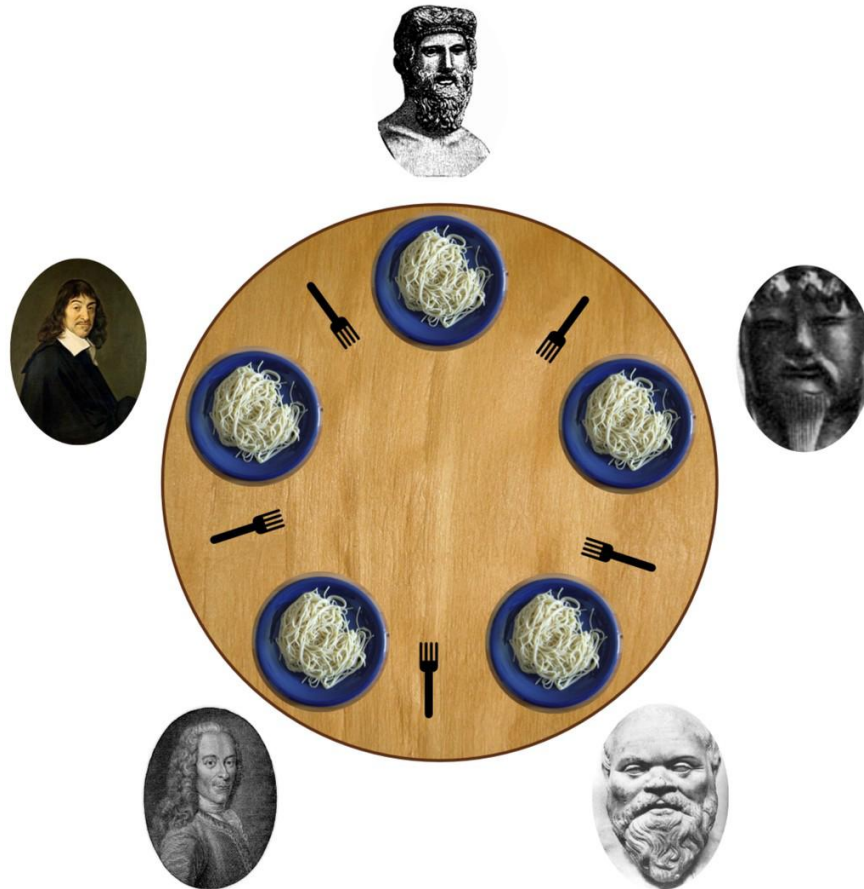
- *erőforrásokért vetélkedés közben,*
- *eseményekre várakozáskor, vagyis, amikor kölcsönös kizárást használunk.*

(Mindenképpen **egynél több kölcsönös kizárást** igénylő erőforrásról van szó)

Holtpont - Ebédelő filozófusok - példa

Klasszikus példa: **Ebédelő filozófusok** esete

Lásd. jegyzet!



Holtpont - Ebédelő filozófusok - példa

Ez a *holtpont* kialakulásának és az *elkerülésének* klasszikus példája.

- *A probléma a következő:* öt filozófus ül egy asztal körül, mindegyiknek van *tányér spagettije* és mindegyik tányér mellett van egy villa.
- Spagettit enni viszont csak két villával lehet.
- A filozófusok felváltva *esznek* vagy *gondolkodnak*.

Holtpont - Ebédelő filozófusok - példa

- Amikor egy filozófus megéhezik, megpróbálja megszerezni a *bal és jobb villát*, bármilyen sorrendben, de egymás után.
- Ha sikerül két villát szereznie, akkor eszik egy darabig, majd leteszi a villákat és folytatja a gondolkodást.
- A kulcskérdés: tudnánk-e írni olyan programot - minden filozófus számára -, ami megoldja a problémát és sohasem okoz holtpontot, egyik sem hal éhen, mindegyik tud gondolkodni is.

Holtpont - Ebédelő filozófusok - példa

Jó megoldást (szerző: Tannenbaum).

- Ebben a *szemafor* védi a villákkal való manipulációkat: egyszerre csak egy filozófus vehet-tehet villákat!
- Másrészt számon tartjuk a filozófusok állapotát: ÉHES, ESZIK, GONDOLKODIK állapotokat.
- Az algoritmus 5 filozófus esetén megengedi, hogy egyszerre két filozófus egyen!

Holtpont - kialakulásának feltételei

A holtpont kialakulásának feltételei:

- 1.** Kölcsönös kizárás.
- 2.** Foglalta várakozás.
- 3.** Nincs erőszakos erőforrás-elvétel.
- 4.** Körkörös várakozás.

Holtpont - kialakulásának szükséges feltételei

- 1. Kölcsönös kizárás:** legyenek olyan erőforrások a rendszerben, amelyeket a *processzek csak kizárólagosan használhatnak*.
- 2. Foglalta várakozás:** legyen olyan processz, amelyik *lefoglalta tart erőforrásokat, közben más erőforrásokra várakozik*.

Holtpont - kialakulásának szükséges feltételei

3. *Nincs erőszakos erőforrás-elvétel:* a rendszerben minden processz addig birtokolja a megkapott erőforrásokat, amíg fel nem szabadítja azokat.

4. *Körkörös várakozás:* a rendszerben lévő processzek közül létezik egy olyan $\{P_0, P_1, \dots, P_n\}$ sorozat, amelyben P_0 egy P_1 által lefoglalva tartott erőforrásra vár, P_i egy P_{i+1} -re, végül P_n , pedig P_0 -ra vár.

Holtpont – kezelési fajták

Kezelési fajták

- 1.** Strucc algoritmus.
- 2.** Detektálás, megszüntetés.
- 3.** Megelőzés.

Holtpont - strucc algoritmus

Strucc algoritmus

- Nem törődünk a problémával, mert igen kicsi az előfordulási esélye.

Előnye: Nulla költségigény.

Hátránya: ha mégis előfordul, akkor előbb utóbb „lefagy” a gép...

- Kritikus rendszerekben nem alkalmazható.

Holtpont - strucc algoritmus

Strucc algoritmus: nem veszünk tudomást a problémáról és nem teszünk semmit.

- *Bizonyos típusú rendszereknél* (nagy kockázat) nem engedhető meg.
- *Megengedhető kisebb kockázatú* rendszereknél.
- Mérlegelni kell a *probléma súlyát* és a *megoldás árát*.

Holtpont - Detektálás

Detektálás és megszüntetés (észlel, észrevesz, felfedez/feloldás)

Detektálás mikor?

A minél *hamarabbi detektálás* fontos, mert,

- a holtpontban levő *processzek-et valószínűleg fel kell áldozni*, és
- az idővel növekszik a holtpontban levők száma.

Holtpont - Detektálás

a.) *Rendszeres időközönként:*

Gyakran: hamarabb detektál, nagyobb költségigény.

Ritkán: később detektál, kisebb költségigény.

b.) *Eseményhez kötötten:*

Minden erőforrás foglалás után: túl nagy költségigény.

Holtpont – Detektálás

Detektálás hogyan?

a.) *Egypéldányos* erőforrások esetén:

Körkörösség figyelésével.

b.) *Többpéldányos* erőforrások esetén:

A kielégíthető processzeket megkereső algoritmussal.

Holtpont – megszüntetés

Megszüntetés hogyan?

- 1.** *Erőforrás elvételével: következmény a processz maradandó sérülése.*
- 2.** *Processz leállításával*
 - a.)** *Összes holtpontban levő processz leállításával:*

Biztos, hogy elegendő, de túl drasztikus.

Holtpont – megszüntetés

b.) *Egyesével leállítani és újra detektálni:*

Költségigényes, plusz döntési algoritmust igényel (áldozat kiválasztás).

Legkevesebbet futott, legtöbb erőforrást birtokló, stb.?

Holtpont – megszüntetés

3. Kármentés - *mentési pontokat*

Nagyobb biztonságot igénylő rendszerek esetén lehetséges a processzekhez közbenső *mentési pontokat* (checkpoint) tárolni.

Nagy költségigény!

Holtpont – megelőzés

I. Megelőzés valamelyik holtpont kialakulási feltétel kiiktatásával.

Nincs teljes körű megoldás

1. feltétel: *kölcsönös kizárás*

Nem iktatható ki, de pl. a *spooling technikával csökkenthető a kölcsönös kizárást igénylő* erőforrások száma.

Holtpont – megelőzés

2. feltétel: *foglalva várakozás*

- *Egy processz egyszerre kell kérje az összes erőforrását: túl nagy hatékonyság romlás.*

3. feltétel: *nincs erőszakos erőforrás elvétel*

- *Csak menthető állapotú erőforrásokra oldható meg.*

4. feltétel: *körkörösség kialakulása*

- *Sorrendi foglалás: csak bizonyos erőforrásokra és programozási szabályok betartásával oldható meg.*

Holtpont – megelőzés

II. Megelőzés állandó mérlegeléssel:

- Minden *erőforrás* *kérés*kor megvizsgálni, hogy a *kérés* teljesítésekor biztonságos állapotban marad-e a rendszer (azaz nem alakulhat ki holtpont)

Lásd:

(Bankár algoritmus - Ez a megoldás sem használható minden erőforrás esetén)

Holtpont – kezelés tanulságai

Az OS a saját erőforrásaira, saját kódjaiban képes megoldani a holtpont megelőzését (pl. használja a sorrendi foglalást saját kódjaiban), de nem *tud olyan mechanizmust biztosítani*, hogy a *felhasználók szoftvereire* is elkerülhető legyen a holtpont.

Egy általános célú OS-nel nem éri meg használni a *detektálást, megszüntetést* (költség vs. nyereség).

Holtpont – kezelés tanulságai

Mit jelent ez?

Vagyis a **programozóknak saját maguknak kell ügyelni** arra, hogy programjaikat úgy készítsék, hogy mikor *kölcsönös kizárásokat használnak ne alakulhasson ki holtpont.*”

Bankár algoritmus – holdpont elkerülése

„A holdpont elkerülésének módszere:

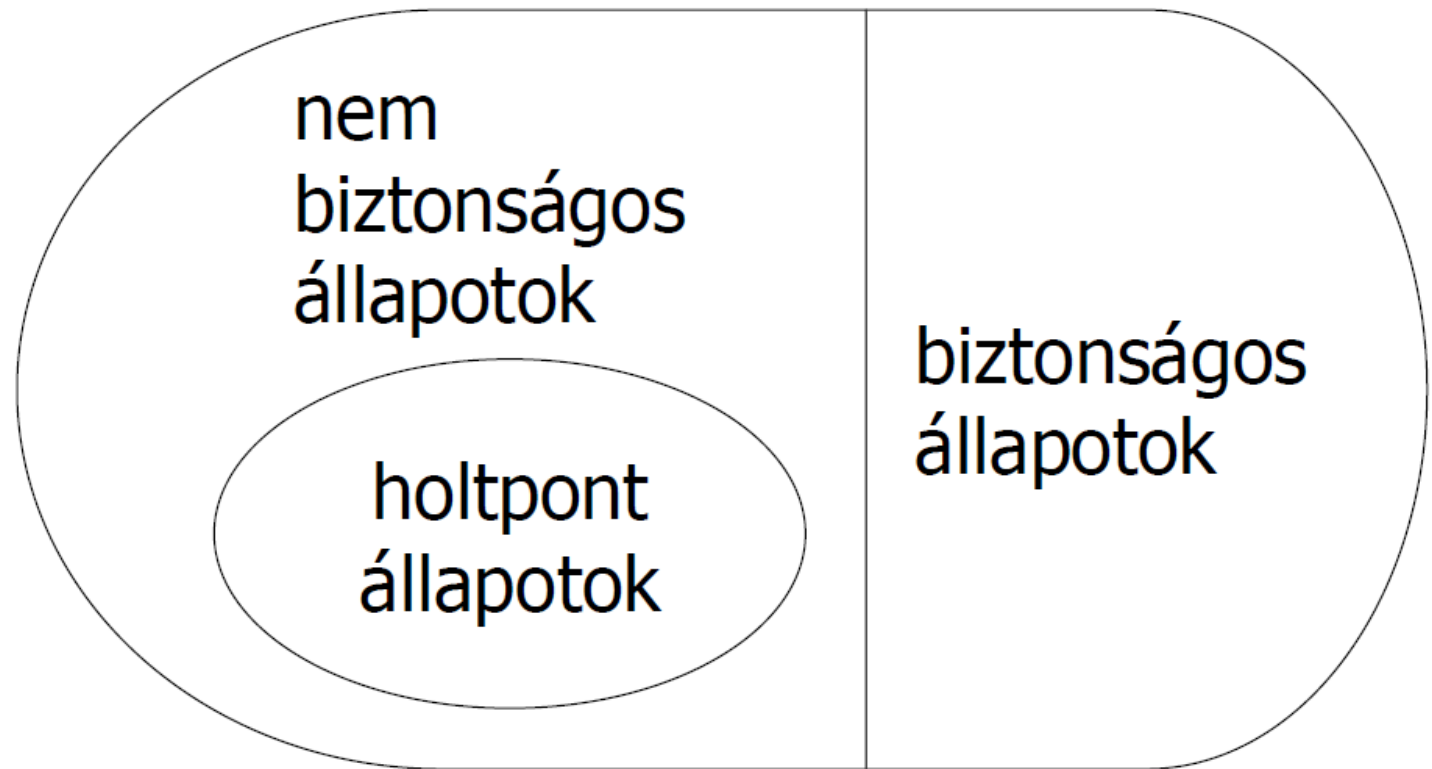
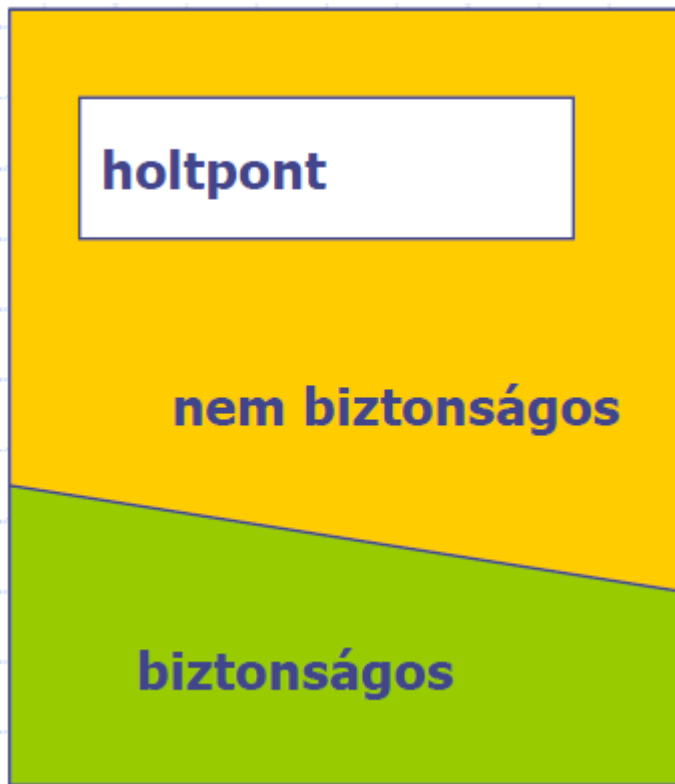
- Az erőforrás igényt csak akkor teljesítjük, ha az így kialakult rendszer *biztonságos állapotban* marad.
- *Biztonságos állapot:*

Található olyan *processz végrehajtási sorrend*, hogy az *aktuálisan futó processz maximális igénye kielégíthető*.

Megvalósítás: Bankár algoritmus.

Bankár algoritmus

Biztonságos és holtpont állapotok viszonya



Forrás: HorváthZS

Bankár algoritmus

- *Biztonságos állapotban nem alakul ki holtpont.*
- *Egy nem biztonságos állapot még nem garantálja holtpont kialakulását* – ez csupán a nem biztonságos állapotok halmazának részhalmazai.
- Egy stratégiai lehetőség a *holtpont elkerülésére*, ha csak biztonságos állapot létrejöttét engedjük meg (banker algorithm).

Bankár algoritmus

Bankár algoritmus:

- Csak azokat a processzeket engedi futni, amelyek *erőforrásigénye kielégíthető*, ha az *összes erőforrást igényli amire szüksége van*.

Bankár algoritmus

A bankok működésének az analógiájára épül.

- A bankvilágban egy bank soha ne kölcsönözhesse ki az összes pénzkészletét.
- Amikor *egy új processz lép a rendszerbe*, annak meg kell adni minden erőforrástípusból az általa *maximálisan kérhető darabszámot*, ami nem lehet több, mint a *rendszer erőforrásainak a teljes száma*.

Bankár algoritmus

- Amikor a processz kéri a konkrét erőforrások egy halmazát, akkor az OS meghatározza, hogy a *kért erőforrások allokálása biztonságos állapotban* hagyja-e a rendszert:
 - *ha igen, akkor a processz megkapja az erőforrást,*
 - *ha nem, várakozni kényszerül.*

Bankár algoritmus - lépések

0. *Először:* Meg kell adni az *össz-erőforrások számát!!!*
1. Felírjuk a processzek által *maximálisan igényelt* erőforrások számait tartalmazó mátrixot. – **MAX.IGÉNY.**
2. Felírjuk a processzek által *lefoglalva tartott* erőforrások számait tartalmazó mátrixot. – **FOGLAL.**
3. A **MAX.IGÉNY**-ből kivonjuk a **FOGLAL** mátrixot, így megkapjuk a *még kielégítetlen igényeket leíró mátrixot*- ez lesz **IGÉNY** mátrix.

Bankár algoritmus - lépések

4. Erőforrás fajtánként összeadjuk a **FOGLALVA *tartott erőforrások* számait**, majd ezeket kivonva az *egyes erőforrások összedarab erőforrások számából* kapjuk meg a *pillanatnyilag rendelkezésre álló erőforrások készletét* – **KÉSZLET**.

5. Megnézzük, hogy a kapott **KÉSZLET**-ből kielégíthető-e valamelyik *processz igénye* – ezt a **IGÉNY** mátrixból nézzük meg.

6a. *Ha nincs ilyen processz*, a rendszer NEM BIZTONSÁGOS állapotba van.

Bankár algoritmus - lépések

6b. Ha van ilyen processz, kielégítjük az igényét.

7. A kiválasztott processz a lefutása után felszabadítja az összes általa használt erőforrást (ez a **FOGLALT mátrix**), azaz **KÉSZLET** új értékét kapjuk, ha az előző értékéhez hozzáadjuk a processz által eredetileg lefoglalva tartott erőforrások számát – **FOGLAL** megfelelő sorát.

8. Visszamegyünk az **5. lépésre**, majd folytatjuk.”

[1][2][3][4][5]

Irodalom

[1] Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

[2] Vincze Dávid: Operációs rendszerek, ME, 2019.

[3] Operációs rendszerek

<https://dcs.uni-pannon.hu/oktatas/tantargyak/operacios-rendszerek>

[4] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

[5] Tannenbaum: Modern Operating Systems, Prentice Hall, 1992.